

MÓDULO 4: Colecciones y Manejo de Errores Idiomático

Aquí conectamos Ownership + Enums (Módulos 2 y 3) con las herramientas que usarás en el 100% del código Rust real: colecciones dinámicas y el sistema de errores basado en `Result<T, E>`.

4.1 `Vec<T>`: Arrays Dinámicos

```
fn main() {
    // Creación
    let mut v: Vec<i32> = Vec::new();
    v.push(1);
    v.push(2);
    v.push(3);
    let v2 = vec![10, 20, 30]; // macro más idiomática

    // Acceso: dos formas, con consecuencias distintas
    let tercero = v[2];           // panic si el índice no existe
    let tercero_opt = v.get(2);    // devuelve Option<i32>, NUNCA
panic
    println!("{}", tercero);
    match tercero_opt {
        Some(val) => println!("Existe: {}", val),
        None => println!("No existe"),
    }

    // Iteración
    for i in &v {                  // &v: préstamo inmutable, no mueve v
        print!("{}", i);
    }
    println();

    let mut v3 = vec![1, 2, 3];
    for i in &mut v3 {             // préstamo mutable: modifica
in-place
        *i *= 10;                  // *i: desreferenciar para
modificar el valor
    }
    println!("{:?}", v3); // [10, 20, 30]
    println!("{:?}", v2);
}
```

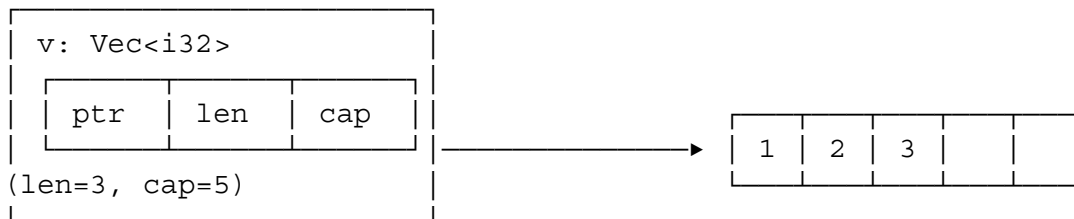
Diagrama mental: `Vec<T>` en memoria

 Diagrama Mental: Distribución de Memoria `Vec<T>`

STACK (LIFO, alto rendimiento)

HEAP (Buffer

dinámico reassignable)



len = cuántos elementos hay realmente. cap = cuánto espacio hay reservado. Cuando len alcanza cap, Rust reserva un bloque nuevo más grande (normalmente el doble), copia los datos, y libera el bloque viejo — esto es *amortized O(1)* para push.

⚠ Error típico del compilador #1: Borrow checker + Vec

```
fn main() {
    let mut v = vec![1, 2, 3];
    let primero = &v[0]; // préstamo inmutable
    v.push(4);           // ❌ préstamo mutable mientras existe uno
                          // inmutable
    println!("{}", primero);
}
```

```
error[E0502]: cannot borrow `v` as mutable because it is also
borrowed as immutable
--> src/main.rs:4:5
3 |     let primero = &v[0];
  |                   ---- immutable borrow occurs here
4 |     v.push(4);
  |     ^^^^^^^ mutable borrow occurs here
5 |     println!("{}", primero);
  |                   ----- immutable borrow later used here
```

Por qué esto NO es una limitación arbitraria: push puede provocar una **reasignación de memoria** (si len == cap, Rust mueve todo el buffer a otra dirección). Si primero siguiera apuntando a la dirección vieja, sería un **dangling pointer** — el bug clásico de "iterator invalidation" en C++ que causa corrupción de memoria silenciosa. Rust lo convierte en un error de compilación en vez de un crash aleatorio en producción.

4.2 HashMap<K, V>

```
use std::collections::HashMap;
```

```
fn main() {
    let mut inventario: HashMap<String, i32> = HashMap::new();
    inventario.insert(String::from("manzanas"), 50);
    inventario.insert(String::from("peras"), 30);

    // Acceso seguro
```

```

match inventario.get("manzanas") {
    Some(cantidad) => println!("Hay {} manzanas", cantidad),
    None => println!("No hay manzanas registradas"),
}

// entry API: patrón idiomático para "insertar si no existe, o
// modificar si existe"
inventario.entry(String::from("naranjas")).or_insert(0);
*inventario.entry(String::from("manzanas")).or_insert(0) += 10;

// Iteración (orden NO garantizado)
for (clave, valor) in &inventario {
    println!("{}", clave, valor);
}

// Contador de palabras: el patrón entry API más clásico
let texto = "el rapido zorro salta el zorro duerme";
let mut contador: HashMap<&str, i32> = HashMap::new();
for palabra in texto.split_whitespace() {
    *contador.entry(palabra).or_insert(0) += 1;
}
println!("{}", contador);
}

```

Nota de ownership: insert mueve tanto la clave como el valor hacia el HashMap (si no son Copy). Si necesitas seguir usando el String original después de insertarlo, tendrás que `.clone()` antes.

4.2.b Colecciones Especializadas: HashSet, BTreeMap y VecDeque

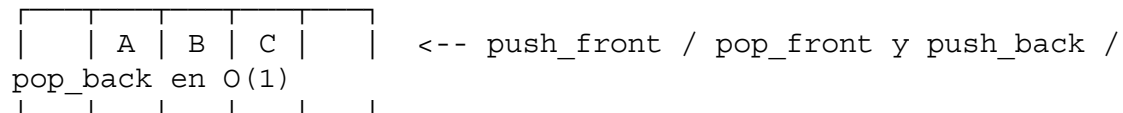
Además de `Vec<T>` y `HashMap<K, V>`, la biblioteca estándar provee estructuras optimizadas para distintos patrones de acceso y ordenamiento:

1. **HashSet<T>**: Conjunto de elementos únicos no ordenados basados en hashing. Ideal para deduplicación y chequeos de pertenencia en $O(1)$ promedio.
2. **BTreeMap<K, V>**: Mapa ordenado por sus claves basado en árboles B. Permite iteración ordenada y consultas por rangos (`.range()`) con complejidad $O(\log n)$.
3. **VecDeque<T>**: Buffer circular de doble extremo. Permite inserciones y eliminaciones eficientes tanto al inicio como al final en $O(1)$ amortizado (`push_front`, `pop_front`, `push_back`, `pop_back`).

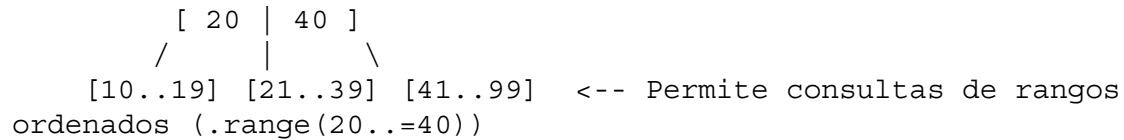


Diagrama Mental: Comparativa de Estructuras

VecDeque<T> (Buffer Circular):



BTreeMap<K, V> (Árbol B Ordenado):



Código Compilable

```
use std::collections::{BTreeMap, HashSet, VecDeque};
fn main() {
    // 1. HashSet: Pertenencia y Deduplicación
    let mut ips_bloqueadas: HashSet<String> = HashSet::new();
    ips_bloqueadas.insert("192.168.1.10".into());
    ips_bloqueadas.insert("10.0.0.5".into());
    ips_bloqueadas.insert("192.168.1.10".into()); // Duplicado
    ignorado
    println!("IPs bloqueadas (únicas): {:?}" , ips_bloqueadas.len());
    // 2
    // 2. BTreeMap: Ordenamiento por Clave y Rangos
    let mut metricas_por_timestamp: BTreeMap<u64, f64> =
BTreeMap::new();
    metricas_por_timestamp.insert(1700000000, 45.2);
    metricas_por_timestamp.insert(1700000010, 89.1);
    metricas_por_timestamp.insert(1700000005, 62.0);

    println!("Métricas ordenadas cronológicamente:");
    for (ts, val) in &metricas_por_timestamp {
        println!("  TS {}: {}%", ts, val); // Imprime en orden: 00,
05, 10
    }

    // Consulta de rango en BTreeMap
    let rango =
metricas_por_timestamp.range(1700000000..=1700000005);
    println!("Rango específico: {:?}" , rango.collect::<Vec<_>>());

    // 3. VecDeque: Cola FIFO/LIFO de Alto Rendimiento
    let mut cola_mensajes: VecDeque<&str> = VecDeque::new();
    cola_mensajes.push_back("msg_1");
    cola_mensajes.push_back("msg_2");
    cola_mensajes.push_front("msg_urgente_0");

    while let Some(msg) = cola_mensajes.pop_front() {
        println!("Procesando: {}", msg);
    }
}
```

4.2.c Mutación Eficiente de Colecciones: `.drain()` y `.retain()`

Modificar o filtrar colecciones grandes *in-place* sin incurrir en realocaciones de memoria es un requisito crítico en sistemas de alto rendimiento:

- **`.retain(predicate)`**: Filtra la colección modificándola *in-place*. Conserva únicamente los elementos que cumplen la condición booleana, liberando la memoria de los descartados sin realocar el buffer principal.
- **`.drain(rango)`**: Extrae un rango de elementos transfiriendo su *ownership* a través de un iterador, dejando el buffer de la colección intacto para reutilizar su capacidad.

Código Compilable

```
fn main() {
    // 1. .retain(): Filtrado in-place sin copiar memoria
    let mut sesiones = vec![
        ("usr_1", true),
        ("usr_2", false),
        ("usr_3", true),
        ("usr_4", false),
    ];

    // Retener solo las sesiones activas
    sesiones.retain(|(_id, activa)| *activa);
    println!("Sesiones activas post-retain: {:?}", sesiones);

    // 2. .drain(): Extracción por batches reutilizando capacidad
    let mut cola_eventos = vec!["evt_1", "evt_2", "evt_3", "evt_4",
    "evt_5"];
    let capacidad_inicial = cola_eventos.capacity();

    // Extraemos los primeros 3 elementos con ownership
    let batch: Vec<&str> = cola_eventos.drain(0..3).collect();
    println!("Batch extraído: {:?}", batch); // ["evt_1", "evt_2",
    "evt_3"]
    println!("Cola restante: {:?}", cola_eventos); // ["evt_4",
    "evt_5"]
    assert_eq!(cola_eventos.capacity(), capacidad_inicial); //
    Capacidad intacta, cero realloc
}
```

4.3 String vs `&str`: La Distinción que Confunde a Todos

```
fn main() {
    let s1: &str = "literal";           // vive en el binario
    compilado (.rodata), &str
    let s2: String = String::from("propio"); // vive en el heap, es
    owned
```

```

    let s3: &str = &s2;                                // referencia (slice) al
String
    let s4: String = s1.to_string();                    // conversión &str -> String
(copia al heap)

    // Concatenación
    let s5 = s2 + " mundo"; // NOTA: s2 se MUEVE aquí (ver por qué
abajo)
    println!("{}", s5);
    // println!("{}", s2); // ❌ s2 fue movido

    let s6 = format!("{}", s1, s3); // format! nunca mueve,
siempre es el idiomático
    println!("{}", s6);
}

```

Por qué + mueve el operando izquierdo: el operador + en String está implementado con la firma `fn add(self, other: &str) -> String` — toma `self` **por valor**. Internamente reutiliza el buffer del heap de `s2` y le hace `push_str`, evitando una copia extra. Es una optimización de rendimiento expuesta directamente en la firma del tipo. Por esto **format! es preferible** en el 90% de los casos: es más legible y no mueve nada.

🧠 Diagrama mental: por qué no puedes indexar un String con `s[0]`

```

fn main() {
    let s = String::from("hóla");
    // let c = s[0]; // ❌ no compila
    let c = s.chars().nth(0); // ✅ forma correcta
    println!("{}", c);
}

```

String es UTF-8. El carácter ó ocupa **2 bytes**, no 1. `s[0]` sería ambiguo: ¿el primer byte (que podría partir un carácter a la mitad) o el primer carácter Unicode? Rust se niega a adivinar y **no implementa indexación directa** en String, forzándote a ser explícito sobre si quieres bytes (`.as_bytes()`), caracteres (`.chars()`), o grafemas (requiere un crate externo, porque incluso "carácter" es ambiguo en Unicode con emojis compuestos).

4.3.b Closures e Iteradores Profundos: Traits Fn, FnMut, FnOnce y Pipelines

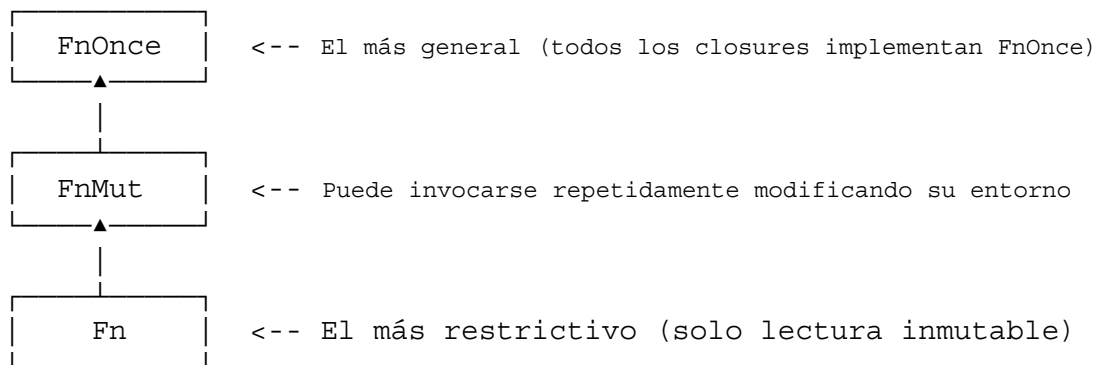
En Rust, los *closures* son funciones anónimas que pueden capturar variables de su entorno léxico. El compilador genera automáticamente una estructura interna para cada closure e implementa uno o más de los tres traits de llamada según cómo use los valores capturados:

♦ Los Tres Traits de Closures

Trait	Semántica de Captura	Firma interna	Capacidad de Invocación
FnOnce	Consume los valores capturados por valor (<i>Move</i>).	<code>fn call_once(self, args)</code>	Solo una vez (consume su estado).

Trait	Semántica de Captura	Firma interna	Capacidad de Invocación
FnMut	Modifica los valores capturados por referencia mutable.	fn call_mut(&mut self, args)	Múltiples veces mutando su estado interno.
Fn	Solo lee los valores capturados por referencia inmutable.	fn call(&self, args)	Múltiples veces concurrentes o inmutables.

🧠 Diagrama Mental: Jerarquía de Traits de Closures



Código Compilable: Closures y Pipelines Funcionales

```

fn ejecutar_una_vez<F>(f: F)
where
    F: FnOnce() -> String,
{
    println!("FnOnce: {}", f());
}

fn ejecutar_mutador<F>(mut f: F)
where
    F: FnMut(i32),
{
    f(10);
    f(20);
}

fn main() {
    // 1. Demostración de Traits de Closures
    let texto = String::from("datos_consumidos");
  
```

```

// FnOnce: mueve `texto` hacia adentro del closure
let closure_once = move || format!("Consumido: {}", texto);
ejecutar_una_vez(closure_once);

let mut acumulador = 0;
// FnMut: muta la variable capturada por referencia
let closure_mut = |x| acumulador += x;
ejecutar_mutador(closure_mut);
println!("Acumulador post FnMut: {}", acumulador); // 30

// 2. Pipelines de Iteradores Funcionales (Lazy Evaluation)
let numeros = vec![1, 2, 3, 4, 5, 6, 7, 8, 9, 10];

let resultado: Vec<i32> = numeros
    .iter()
    .filter(|&x| x % 2 == 0)           // Solo pares
    .map(|&x| x * 10)                 // Multiplica por 10
    .skip(1)                          // Salta el primero (20)
    .take(3)                          // Toma hasta 3
    .collect();                       // Materializa en un nuevo Vec

println!("Pipeline funcional: {:?}", resultado); // [40, 60, 80]
}

```

4.4 Result<T, E>: Errores como Valores, No como Excepciones

```

enum Result<T, E> {
    Ok(T),
    Err(E),
}

fn dividir(a: f64, b: f64) -> Result<f64, String> {
    if b == 0.0 {
        Err(String::from("división por cero"))
    } else {
        Ok(a / b)
    }
}

fn main() {
    match dividir(10.0, 2.0) {
        Ok(valor) => println!("Resultado: {}", valor),
        Err(e) => println!("Error: {}", e),
    }

    // unwrap: extrae el Ok, o PANIC si es Err (úsalo solo cuando
    SABES que no fallará, o en prototipos)
    let v = dividir(10.0, 5.0).unwrap();
    println!("{}", v);

    // expect: como unwrap, pero con mensaje de error custom (mejor

```

```
para debugging)
    let v2 = dividir(10.0, 5.0).expect("la división no debería
fallar aquí");
    println!("{}", v2);
}
```

4.5 El Operador ?: Propagación Idiomática de Errores

Este operador es, junto con Ownership, lo que define el "sabor" del código Rust real.

```
use std::fs::File;
use std::io::{self, Read};
fn leer_archivo(ruta: &str) -> Result<String, io::Error> {
    let mut archivo = File::open(ruta)?; // si falla, RETORNA el Err
inmediatamente
    let mut contenido = String::new();
    archivo.read_to_string(&mut contenido)?; // mismo patrón
    Ok(contenido)
}
fn main() {
    match leer_archivo("no_existe.txt") {
        Ok(contenido) => println!("Contenido: {}", contenido),
        Err(e) => println!("Error al leer: {}", e),
    }
}
```

Diagrama mental: qué hace ? exactamente

```
// Esto:
let contenido = archivo.read_to_string(&mut buffer)?;

// Es azúcar sintáctico para esto:
let contenido = match archivo.read_to_string(&mut buffer) {
    Ok(valor) => valor,
    Err(e) => return Err(From::from(e)), // convierte el error si hace
falta, y retorna
};
```

? solo funciona dentro de una función que devuelve Result (o Option). Es la razón por la que casi toda función en Rust que puede fallar tiene la firma -> Result<T, E>.

Error típico del compilador #2: Usar ? fuera de una función Result

```
fn main() {
    let archivo = std::fs::File::open("x.txt"?); // ❌
}
```

error[E0277]: the `?` operator can only be used in a function that returns
`Result` or `Option`

```
--> src/main.rs:2:44
|
1 | fn main() {
|   ---- this function should return `Result` or `Option` to accept `?`
2 |     let archivo = std::fs::File::open("x.txt"?);
```

| ^ cannot use the `??` operator
Solución: cambiar `fn main()` por `fn main() -> Result<(), Box<dyn std::error::Error>>` (patrón muy común para binarios pequeños/CLIs), o manejar el error con `match/unwrap` dentro de `main`.

4.5.b Combinadores Avanzados de Result (`map_err`, `and_then`, `context`)

Encadenar operaciones propensas a error sin anidar bloques `match` es la marca distintiva del código idiomático en Rust:

1. **`.map_err(fn)`:** Transforma el tipo de error en caso de `Err`, dejando el valor `Ok` intacto.
2. **`.and_then(fn)`:** Encadena una operación que retorna otro `Result`, aplanando el resultado (evita anidamientos `Result<Result<T, E>, E>`).
3. **`.context("mensaje")`** (vía `anyhow`): Añade contexto estructurado a la cadena de errores antes de propagar con `?`.

```
use anyhow::{Context, Result};
```

```
#[derive(Debug)]
enum ErrorValidacion {
    CadenaVacía,
    FueraDeRango(u32),
}

fn validar_no_vacio(input: &str) -> std::result::Result<&str,
ErrorValidacion> {
    if input.trim().is_empty() {
        Err(ErrorValidacion::CadenaVacía)
    } else {
        Ok(input)
    }
}

fn parsear_id(input: &str) -> std::result::Result<u32, ErrorValidacion> {
    input
        .trim()
        .parse::<u32>()
        .map_err(|_| ErrorValidacion::FueraDeRango(0))
        .and_then(|id| {
            if id >= 100 {
                Ok(id)
            } else {
                Err(ErrorValidacion::FueraDeRango(id))
            }
        })
}

fn pipeline_completo(input: &str) -> Result<u32> {
    validar_no_vacio(input)
        .map_err(|e| anyhow::anyhow!("Fallo de validación: {:?}", e))
        .and_then(|s| parsear_id(s).map_err(|e| anyhow::anyhow!("ID
inválido: {:?}", e)))
        .context("Error crítico al procesar la entrada de usuario")
}

fn main() {
    match pipeline_completo("250") {
        Ok(id) => println!("ID procesado exitosamente: {}", id),
        Err(e) => println!("Error con contexto acumulado:\n{:?}", e),
    }
}
```

4.6 Errores Personalizados: El Enum como Taxonomía de Fallos

```
use std::fmt;

#[derive(Debug)]
enum ErrorApp {
    NoEncontrado(String),
    Invalido { campo: String, razon: String },
    ErrorIO(String),
}

impl fmt::Display for ErrorApp {
    fn fmt(&self, f: &mut fmt::Formatter) -> fmt::Result {
        match self {
            ErrorApp::NoEncontrado(id) => write!(f, "No se encontró: {}",
id),
            ErrorApp::Invalido { campo, razon } => write!(f, "Campo '{}'
inválido: {}", campo, razon),
            ErrorApp::ErrorIO(msg) => write!(f, "Error de I/O: {}", msg),
        }
    }
}

impl std::error::Error for ErrorApp {} // habilita interoperar con Box<dyn
Error>

fn buscar_usuario(id: u32) -> Result<String, ErrorApp> {
    if id == 0 {
        return Err(ErrorApp::Invalido {
            campo: String::from("id"),
            razon: String::from("no puede ser 0"),
        });
    }
    if id > 100 {
        return Err(ErrorApp::NoEncontrado(format!("usuario#{}", id)));
    }
    Ok(format!("Usuario #{}", id))
}

fn main() {
    for id in [0, 50, 200] {
        match buscar_usuario(id) {
            Ok(u) => println!("{}", u),
            Err(e) => println!("{}", e), // usa Display gracias al impl
de arriba
        }
    }
}
```

Esto es exactamente lo que hace el `crate thiserror` por debajo, pero con macros para eliminar el boilerplate.

4.7 thiserror: Errores de Librería (tipados)

[dependencies]

```

thiserror = "1.0"

use thiserror::Error;

#[derive(Error, Debug)]
enum ErrorApp {
    #[error("No se encontró: {0}")]
    NoEncontrado(String),
    #[error("Campo '{campo}' inválido: {razon}")]
    Invalido { campo: String, razon: String },
    #[error("Error de I/O")]
    ErrorIO(#[from] std::io::Error), // #[from] genera conversión
    automática con `?`
}

fn leer_config() -> Result<String, ErrorApp> {
    let contenido = std::fs::read_to_string("config.toml")?; //
    gracias a #[from], convierte io::Error -> ErrorApp automáticamente
    Ok(contenido)
}

fn main() {
    match leer_config() {
        Ok(c) => println!("{}", c),
        Err(e) => println!("Error: {}", e),
    }
}

```

thiserror elimina TODO el boilerplate de impl Display y impl Error manual del punto 4.6, generándolo con macros derive. **Úsalo cuando escribes una librería** y quieres que los consumidores puedan hacer match sobre variantes específicas de tu error.

4.8 anyhow: Errores de Aplicación (opacos, con contexto)

```

[dependencies]
anyhow = "1.0"

use anyhow::{Context, Result, bail};

```

```
fn procesar_edad(texto: &str) -> Result<u32> {
    let edad: u32 = texto.parse()
        .context("el texto no es un número válido"); // añade
    contexto legible al error
    if edad > 150 {
        bail!("edad {} no es realista", edad); // atajo para return
    }
    Err(anyhow!(...))
    Ok(edad)
}

fn main() -> Result<()> {
    let edad = procesar_edad("abc"); // el error se propaga con
    TODO el contexto acumulado
    println!("{}", edad);
    Ok(())
}
```

♦ Tabla de decisión: thiserror vs anyhow

Escenario	Usa
Estás escribiendo una librería que otros van a consumir y necesitan hacer match sobre tipos de error específicos	thiserror
Estás escribiendo una aplicación/binario (CLI, servidor) donde solo te importa mostrar/loggear el error, no diferenciarlo por tipo	anyhow
Necesitas añadir contexto legible ("fallo al conectar a X") sobre errores de crates de terceros	anyhow con .context()
Necesitas que el error tenga variantes tipadas y mensajes con Display custom	thiserror

En la práctica, es común ver thiserror en el core/librería de un proyecto y anyhow en el binario que la consume.

4.8.b Resiliencia en Runtime: panic! vs Result y Aislamiento con catch_unwind

Filosofía: Errores Recuperables vs Irrecuperables

1. **Result<T, E>**: Para errores anticipados en runtime (red, parsing, permisos).
2. **panic!**: Para bugs irreversibles del desarrollador (invariantes de memoria violadas, aserciones fallidas).

En servidores web o hilos de procesamiento en batch, un panic! dentro de un request no debe derribar el proceso completo. Rust permite capturar el *stack unwinding* mediante `std::panic::catch_unwind`:

```
use std::panic;
```

```
fn tarea_riesgosa(divisor: i32) -> i32 {
```

```

        if divisor == 0 {
            panic!("División por cero en tarea worker!");
        }
        100 / divisor
    }

fn main() {
    let entradas = vec![5, 0, 2];

    for divisor in entradas {
        // Aislamiento del stack unwind para resiliencia
        let resultado = panic::catch_unwind(|| {
            tarea_riesgosa(divisor)
        });

        match resultado {
            Ok(val) => println!("Operación exitosa: {}", val),
            Err(e) => {
                // El hilo principal sobrevive al pánico
                println!("⚠ Worker atrapó un pánico controlado:
{:?}", e.downcast_ref:::<&str>());
            }
        }
    }

    println!("El servidor principal sigue corriendo sin
interrupciones.");
}

```

⚠ **Advertencia de Producción:** `catch_unwind` solo captura pánicos que realizan *unwinding*. Si el perfil de compilación utiliza `panic = "abort"` en `Cargo.toml`, el proceso finalizará de inmediato a nivel del SO sin posibilidad de captura.

🎯 Ejercicio Integrador — Módulo 4

Proyecto: Sistema de Gestión de Biblioteca con Persistencia en Memoria

Crea cargo new biblioteca con anyhow y thiserror como dependencias.

Requisitos:

1. Define un struct `Libro` { titulo: String, autor: String, isbn: String, disponible: bool }.
2. Define un struct `Biblioteca` { libros: HashMap<String, Libro> } (clave = ISBN).
3. Define un enum de error con `thiserror`:

```

#[derive(Error, Debug)]
enum ErrorBiblioteca {
    #[error("Libro con ISBN {0} no encontrado")]
    NoEncontrado(String),
    #[error("El libro '{0}' ya está prestado")]
    YaPrestado(String),
}

```
4. Implementa en `impl Biblioteca`:

- fn nueva() -> Self
 - fn agregar_libro(&mut self, libro: Libro)
 - fn prestar_libro(&mut self, isbn: &str) -> Result<(), ErrorBiblioteca> — usa entry/get_mut, retorna NoEncontrado si no existe, YaPrestado si disponible == false, y marca disponible = false si tiene éxito.
 - fn devolver_libro(&mut self, isbn: &str) -> Result<(), ErrorBiblioteca> — análogo, marcando disponible = true.
 - fn buscar_por_autor<'a>(&'a self, autor: &str) -> Vec<&'a Libro> (no te preocupes por el 'a todavía, solo cópialo — lo explicamos a fondo en el Módulo 6; por ahora practica que devuelve un Vec de referencias, no de libros clonados).
5. En main() -> anyhow::Result<()>:
- Agrega 3-4 libros.
 - Presta uno, intenta prestarlo de nuevo (debe fallar), captura el error con match y muéstralo con {} (usa el Display que genera thiserror).
 - Usa el operador ? para propagar un error de "devolver un libro con ISBN inexistente" hasta main, y observa cómo se imprime automáticamente si main retorna Err.

Reto extra: agrega una función que lea un ISBN desde stdin, y usa anyhow::Context para envolver el error de parseo con un mensaje más descriptivo si el usuario ingresa algo inválido.

Ejercicio Integrador 2

Proyecto: Pipeline de Ingesta y Limpieza de Métricas Resiliente

Crea cargo new pipeline_metricas e implementa:

1. **Estructura de Datos:**
 - Almacena registros crudos en un VecDeque<String>.
 - Utiliza un HashSet<String> para ignorar métricas duplicadas.
 - Almacena las métricas procesadas y consolidadas en un BTreeMap<u64, f64> indexadas por timestamp unix.
2. **Filtrado y Batches:**
 - Implementa una función de limpieza que use .retain() para descartar logs mal formateados.
 - Procesa la cola en lotes de 5 elementos usando .drain(..).
3. **Pipeline Funcional y Combinadores:**
 - Construye un pipeline de iteradores con closures (FnMut) que parsee líneas tipo "TIMESTAMP:VALOR" usando combinadores .and_then() y .map_err().
4. **Contención de Pánicos:**
 - Envuelve el procesamiento de cada batch dentro de std::panic::catch_unwind para asegurar que entradas maliciosas no detengan la ingesta global.